



Overview (your backend system)

You're using:

- Node.js → logic layer
- Supabase → DB + auth



PAIR 1: Database (Schema Team)



Their Job

Design the **structure** of your data



What they DO

1. Create tables

- calendars
- collaborators
- events

2. Define relationships

Example:

`events.calendar_id → calendars.id`

`collaborators.calendar_id → calendars.id`

3. Add constraints

- NOT NULL
- foreign keys
- default values

4. Maintain schema consistency

- naming conventions
- clean structure



Their Deliverables

- Final table schema
- SQL scripts
- Documentation like:

Table: events

- id
- title
- calendar_id
- created_by

What they should NOT do

- Write APIs
- Write business logic
- Randomly change schema without informing others

They coordinate with:

- API team (field names must match)
- RLS team (policies depend on schema)

PAIR 2: RLS (Security Team)

Their Job

Protect the database using **Row Level Security**

What they DO

1. Write policies

Examples:

- Only collaborators can view events
- Only admins can delete

2. Enable RLS on tables

3. Test access

- try queries as different users

- ensure correct restrictions

4. Debug permission issues

- fix “permission denied” errors



Their Deliverables

- RLS policies for each table
- Test cases:

User A → can view event

User B → cannot view event

✗ What they should NOT do

- Decide business logic alone
- Block APIs unknowingly
- Work without testing



They coordinate with:

- Auth team (user identity)
- API team (queries must work)



PAIR 3: Auth + Authorization Team



Their Job

Handle **who the user is + what they are allowed to do**



What they DO

1. Auth middleware

```
req.user = decodedUser
```

2. Token verification

Using Supabase auth

3. Role logic

- admin vs collaborator
- permissions

4. Helper functions

```
isAdmin(user, calendarId)  
isCollaborator(user, calendarId)
```



Their Deliverables

- working auth middleware
- reusable role-check functions



What they should NOT do

- Direct DB schema changes
- Write full APIs (only support them)



They coordinate with:

- API team (used inside routes)
- RLS team (logic alignment)



PAIR 4: API Development Team (Core Team)



Their Job

Build the **actual backend endpoints**



What they DO

1. Create routes

```
POST /calendars
POST /events
GET /calendars
```

2. Call Supabase

- insert
- fetch
- update

3. Use auth + role checks

```
if (!isAdmin(user)) throw error
```

4. Send responses

```
{ "success": true }
```



Their Deliverables

- working APIs
- tested endpoints



What they should NOT do

- Change DB structure directly
- Ignore RLS issues
- Skip validation



They coordinate with:

- DB team (schema)
- Auth team (middleware)
- RLS team (permissions)



(Optional) PAIR 5: Integration + Testing

(If you want to use remaining people effectively)

Their Job

Make sure everything works together

What they DO

- Test APIs using Postman
- Report bugs
- Help connect frontend

Deliverables

- bug reports
- tested flows

How everything connects (VERY important)

DB Team → defines structure

RLS Team → protects data

Auth Team → identifies user

API Team → executes logic

Real Example (Create Event)

1. API team → builds `/events`
2. Auth team → attaches user
3. API checks role
4. DB team → table exists
5. API inserts into Supabase
6. RLS team → allows or blocks